

CSCI 1430 Final Project Report: A.R.M. (Appendge Replication Machine)

The Boyz: Isabella Szabo, Johnny Kuehnis, Ross Goldbaum
TA name: Lana Yang-Maccini. Brown University

Abstract

We present the Appendage Replication Machine (A.R.M.), a system that uses a Kinect depth camera to capture human hand and arm motion while replicating it on a robotic arm in real time. Our approach integrates joint detection, hand state classification, depth-based localization, and physical actuation through inverse kinematics. The result is a low-latency, accurate, and intuitive interface between human motion and robotic hardware.

1. Introduction

Translating natural human motion into robotic control in real time remains a key challenge in human-to-robot interaction. Motion capture systems certainly exist, but they can often be too expensive or unintuitive for ease of use. Our goal with this project was to design a system that allows for a user to move their arm and hand freely and have a robot hand replicate them as closely as possible. Tackling this task involves several moving parts and a combination of both software and hardware tasks, with a major source of challenge also coming from robust detection in addition to smooth hardware control and low latency. Our approach involves using Microsoft's Kinect Depth Camera to detect the joints, and hand position, estimate its 3D position, determine whether the hand is open or closed, and then transmitting that info to control the hardware.

If this problem were fully solved, it would open up new opportunities in assistive robotics and human-computer interaction. While our project will be more rudimentary, a reliable, low-cost method for capturing and replicating 3D hand motion could be used in teleoperation, gesture-based interfaces, or prosthetics—making robotic technology more accessible and adaptable to real-world human movement.

2. Related Work

Building upon existing research in pose estimation and depth-based 3D localization, we used MediaPipe Pose [3] to detect shoulder, elbow, and wrist joints in real-time. For

the hand state classification, we trained a CNN based on VGG-16 [5] with a custom head, leveraging a dataset of ending in a softmax layer. The model was trained on a dataset of depth camera images showing hands labeled as either open or closed. Preprocessing was performed using the `vgg16.preprocess_input` function from Keras, and the final label was selected based on the highest predicted probability. For Arduino code, we extended functionality from the Adafruit component library

Alternative approaches to joint and hand localization include OpenPose [1] and custom-trained models that require annotated datasets and long training times. We'll discuss these further in the technical discussion in the results section.

3. Method

Our method to accomplishing our task included four primary steps: Hand detection, hand classification, hand position calculation, and translating those to the robot.

First, using MediaPipe Pose, we detect the user's right shoulder (index 12), elbow (index 14), and wrist (index 16) on the rgb image. We then compute the vector from the elbow to the wrist, extending it beyond the wrist to estimate the hand location. We also explored other hand localization techniques, as discussed above, but this was the method that proved to be the most stable and efficient for our needs. Using the bounding box coordinates from the rgb image, we can find the equivalent section of the depth image and send that to the classifier.

As you can see in fig 1., what you end up with are 3 dots that hover over the 3 joints, and a yellow bounding box that's drawn around the extrapolated point. The graph on the left shows the x and y coordinates of the joints, and it updates in real time. These equations below highlight the math that went into estimating the hand direction and drawing the bounding box:

1. Estimate Hand Direction Given:

- Wrist pixel location: $\vec{W} = (w_x, w_y)$
- Elbow pixel location: $\vec{E} = (e_x, e_y)$



Figure 1. Joint Detection and Bounding Box

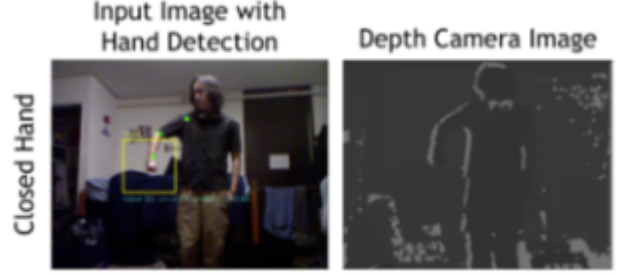


Figure 2. RGB and Depth Stream

Compute the direction vector:

$$\vec{d} = \vec{W} - \vec{E} = (w_x - e_x, w_y - e_y)$$

Project a point past the wrist:

$$\vec{H} = \vec{W} + s \cdot \vec{d} \quad (\text{e.g., } s = 0.45)$$

2. Define Bounding Box

- After finding the center of the hand $\vec{H} = (h_x, h_y)$, define a square bounding box of size L (e.g., $L = 300$ pixels) W, H are the image width and height:

$$x_1 = \max(0, h_x - L/2)$$

$$y_1 = \max(0, h_y - L/2)$$

$$x_2 = \min(W, h_x + L/2)$$

$$y_2 = \min(H, h_y + L/2)$$

Second, once the bounding box around the hand has been computed from the RGB image and aligned to the corresponding region in the depth image, we preprocess that region for classification. Specifically, we crop the depth frame using the bounding box coordinates and resize the resulting patch to match the input dimensions required by our VGG-16-based convolutional neural network. The depth image is normalized and color-mapped to ensure compatibility with the pretrained layers of the base network, which was originally trained on RGB data. Specifically, the depth camera images produced with our depth camera were colored slightly differently than the training data, so some filtering had to be done to match them.

This cropped and resized patch is then passed into our custom classifier. We use a standard VGG-16 architecture as the backbone feature extractor and attach a custom head of fully connected layers to perform binary classification: determining whether the hand is open or closed. The network was trained on the Open/Closed Hand Depth Dataset, which contains 25,200 labeled training images and 10,800 testing images. We applied standard data augmentation techniques such as random rotation, flipping, and slight scaling to increase the robustness of the model to hand orientation and position within the bounding box.

We tuned the learning rate and batch size to ensure stable convergence over 50 epochs. The final model achieved 93.8% training accuracy and 96.3% testing accuracy, showing strong generalization to unseen samples. As shown in the results in ISABELLA PUT WHATEVER IMAGES U WANT, the model maintained consistent performance across different lighting conditions and arm orientations, even in noisy depth frames.

Third, in order to determine the hand's position in 3D space, we used the depth map captured from the Kinect v1 sensor. Within each frame, we cropped the area defined by our dynamically computed bounding box from the raw depth area. Kinect depth images are returned as uint16 arrays where each pixel encodes distance from the sensor in millimeters. As you can see in Figure 2, the Kinect depth camera can return both an rgb stream and a depth camera stream. We used this in the open/closed classification and the hand motion.

After, raw depth values within the bounding box were filtered to remove invalid pixels (0), and the 20 closest depth values were selected. We used the average of these top 20 values to estimate depth, which helped mitigate the effect of background leakage, shadows, and surface noise.

Once the average depth z (in meters) was obtained, we used a simple pinhole camera model to convert the 2D pixel coordinates of the bounding box center into 3D real-world coordinates (x, y, z) . This required knowledge of the camera's intrinsic parameters which we approximated from Kinect specs. The equations below expand on how the average depth was calculated:

Select the closest N values:

$$D' = \text{Top 20 smallest values in } D$$

Average them:

$$z = \frac{1}{N} \sum_{i=1}^N D'_i \quad (\text{in mm})$$

Convert to meters:

$$Z = \frac{z}{1000}$$

Finally, the 3D position and state (open or closed) are sent via serial to an Arduino along with angle values for each servo motor on the arm. Given real-world coordinates (x, y, z) in 3D space and link lengths L_1 (shoulder to elbow) and L_2 (elbow to claw), the joint angles are calculated using inverse kinematics as follows:

Distance to Target in Vertical Plane

$$r = \sqrt{y^2 + z^2}$$

Angles of servos

$$\theta_{\text{base}} = \tan^{-1} \left(\frac{x}{z} \right)$$

$$\theta_{\text{elbow}} = 180^\circ - \cos^{-1} \left(\frac{L_1^2 + L_2^2 - r^2}{2L_1L_2} \right)$$

$$\theta_{\text{shoulder}} = \tan^{-1} \left(\frac{y}{z} \right) + \cos^{-1} \left(\frac{L_1^2 + r^2 - L_2^2}{2L_1r} \right)$$

All angles are then clamped to the mechanical servo limits. PWM signals suitable for the PCA9685 servo driver. Each angle is mapped to a corresponding pulse width, typically ranging from 1000 to 2000 microseconds, which the PCA9685 then sends as 12-bit duty cycle values over I²C. This allows precise positioning of each servo motor based on the inverse kinematics calculation, ensuring the physical arm accurately mirrors the target position in 3D space.

To reduce jitter and prevent sudden, extreme movements in the robotic arm, we apply alpha smoothing to the servo angles before sending commands to the hardware. This method uses exponential smoothing, where each new output angle is a weighted average of the previous output and the newly computed value. Mathematically, this is defined as:

$$\theta_{\text{smoothed}} = (1 - \alpha) \cdot \theta_{\text{prev}} + \alpha \cdot \theta_{\text{new}}$$

To improve stability and responsiveness in the robotic arm, we explored tuning a PID (Proportional-Integral-Derivative) controller on the Arduino to manage servo motion. While typical PWM-driven servos do not natively support closed-loop PID control, applying PID logic in software helps modulate the update rate and pulse targets for smoother transitions, especially when controlling high-torque motors like the MG996R.

The structural components of the robotic arm were fabricated using 1/8-inch black acrylic sheets. Each part was designed in Adobe Illustrator as vector outlines, which were then exported for laser cutting. This process provided precise cuts suitable for assembly and servo integration. The use of acrylic offered a rigid and lightweight frame for the arm without the need for additional machining.

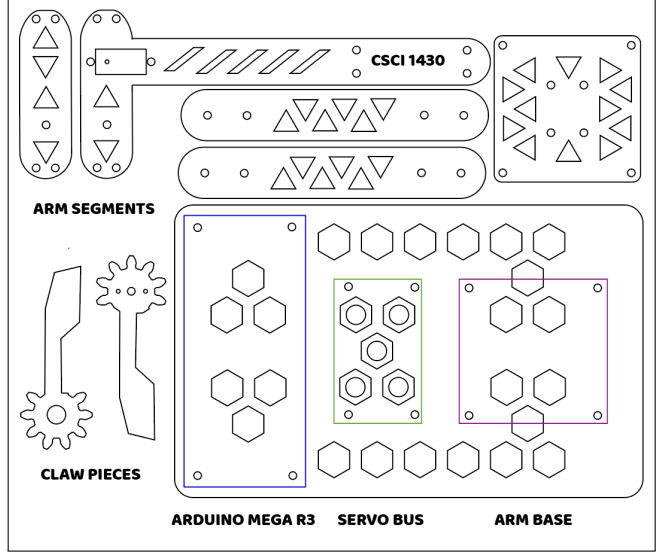


Figure 3. Laser pieces of the arm

4. Results

By the end of the project, we successfully assembled and tested a functional robotic arm capable of mimicking basic human arm movements, allowing the arm to approximate the user's arm orientation with reasonable responsiveness and stability. The final system demonstrated smooth actuation and consistent alignment between virtual input and physical motion. Due to physical constraints in the servo joints, we experienced some jitter and oscillations in the angles. During the last stages of testing, the claw servo broke, but the serial signal still worked. The hand recognition met a 96.8% test accuracy and consistently identified gestures during demos for the public.

We built a virtual simulation of the robotic arm in Unity. This simulation mirrored the arm's kinematic structure and visualized the joint movements in real time using the same inverse kinematics calculations applied in Python. Each angle (base, shoulder, and elbow) was passed into a Unity C# script to update the virtual model's joint transforms.

4.1. Technical Discussion

Developing a reliable and real-time human-to-robot mapping system involved navigating a range of technical trade-offs and failed paths before arriving at our final design. While the core components—joint detection, hand localization, depth-based 3D tracking, and gesture classification—seem modular on the surface, aligning them into a fast and stable pipeline proved to be a complex systems engineering challenge.

One of the biggest challenges early on was hand detection. We explored more “rigorous” pose estimation approaches, such as using OpenPose or training a custom CNN for joint regression on depth images. OpenPose offered detailed body

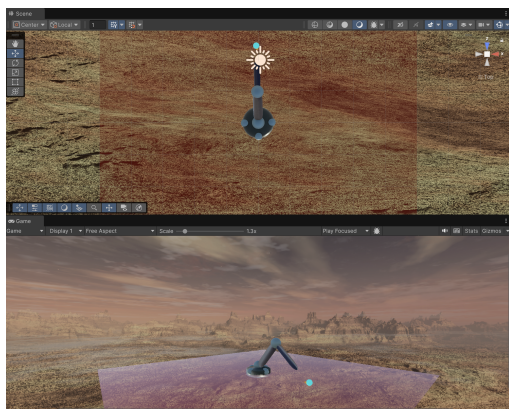


Figure 4. Virtual simulation of arm in Unity

and hand joint tracking, but it was too slow for real-time inference on consumer hardware and too sensitive to visual noise in depth images. As an alternative, we considered building a pose estimator from scratch using a VGG-style CNN trained to predict joint coordinates from depth crops. However, we quickly realized that we lacked access to a high-quality, labeled dataset of depth images with annotated joints. Large-scale datasets like Human3.6M [2], NTU RGB+D [4], and FreiHAND [6] exist, but they were either RGB-only, required multi-camera setups, or lacked the specific joint annotations we needed in the Kinect’s depth domain. Human3.6M in particular sent us down a rabbit hole of trying to get the official dataset, realizing it wasn’t being maintained as well as before (it’d likely take weeks or months to get a response, if at all), then scouring the internet and public github repos to find anyone who had portions of the dataset available for us to use. Creating our own dataset would have required motion capture systems or laborious manual labeling, neither of which were feasible within our timeline, particularly after experimenting with so many different methods.

5. Conclusion

Thus, in this project, we developed a robust and responsive system for translating human hand motion to robotic arm control using a depth camera input. By combining pose estimation/joint detection, 3D localization, classification, and hardware actuation, we demonstrate an accessible method for a gesture-based robotic arm. Our solution remained reliable across various conditions, such as different body types, clothing, background, etc., and maintained solid, real-time performance without relying on expensive or high-power hardware.

This project highlights the effectiveness of combining computer vision software principles with arduino/robotic based hardware. Although we relied on MediaPipe for pose detection, the integration of depth-based spatial inference and a custom-trained classifier enabled us to expand beyond

typical gesture recognition tasks. The final result is a system capable of interpreting and replicating human motion in a way that’s understandable and intuitive to users.

There are many exciting avenues for future development. These include extending the system to track both arms or full-body motion, incorporating fingertip-level hand pose estimation using a separate CNN, or introducing feedback mechanisms from the robotic arm to the user. Enhancing robustness under occlusion and transitioning from camera-relative to world-relative coordinate tracking would also improve generalization. Ultimately, the largest impact of this project is how intuitive it is. You don’t have to be an expert in robotics or operating machinery to operate our device - you just have to move your arm. As such, our work offers a scalable foundation for building more natural human-robot interfaces that merge perception and physical control in real-time environments. Future work could explore full hand articulation and bi-directional feedback from the robotic arm.

6. SRC

Privacy Protection

All data is processed locally and in real time. Neither RGB nor depth frames are stored, transmitted, or logged during system operation. This ensures that no personally identifiable information (PII) is retained and that bystanders are not captured or recorded, thereby mitigating privacy risks and aligning with common data protection principles.

Safety and Gesture Constraints

Our system focuses solely on replicating arm movements: base rotation, shoulder elevation, and elbow extension. It does not reproduce fine-grained hand gestures. Furthermore, joint movement is restricted by both mechanical design and software-imposed angular limits to prevent erratic or unsafe motion. The alpha smoothing and serial error handling eliminates possibilities for a harmful visual input. We recognize the importance of gesture filtering in a more detailed hand model and plan to incorporate additional safety guardrails in future versions.

Fairness and Accessibility

Hand state classification is performed using depth data rather than RGB imagery. This approach abstracts away skin tone and lighting conditions, providing more consistent performance across users. Since depth maps encode structural rather than visual information, the model is less susceptible to bias based on appearance. Our model currently only supports control through the right arm, we aim to include users with a variety of limb geometries to further ensure accessibility.

References

- [1] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Openpose: Realtime multi-person 2d pose estimation using part affinity fields. *arXiv preprint arXiv:1812.08008*, 2018. 1
- [2] Catalin Ionescu, Dragos Papava, Vlad Olaru, and Cristian Sminchisescu. Human3.6m: Large scale datasets and predictive methods for 3d human sensing in natural environments. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(7):1325–1339, 2013. 4
- [3] Camillo Lugaresi, Jiuqiang Tang, Hartwig Nash, Michael McGuire, Valentin Bazarevsky, Yury Zhang, Fan Chang, Ming Guang Yong, and Matthias Grundmann. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019. 1
- [4] Amir Shahroudy, Jun Liu, Tian-Tsong Ng, and Gang Wang. Ntu rgb+d: A large scale dataset for 3d human activity analysis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1010–1019, 2016. 4
- [5] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. 1
- [6] Christian Zimmermann, Duygu Ceylan, Jimei Yang, Bryan Russell, and Thomas Brox. Freihand: A dataset for markerless capture of hand pose and shape from single rgb images. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 813–822, 2019. 4

Appendix

Team contributions

Isabella Szabo led the design, training, and integration of the hand state classification system. She curated a large dataset of depth-based hand images, preprocessing and balancing it to ensure consistent input for model training. Using a modified VGG-16 architecture, she implemented a binary classifier capable of distinguishing between open and closed hands, tuning the model for optimal accuracy and real-time inference.

Johnny Kuehnis was responsible for implementing the pose estimation and hand localization components of the system. He integrated MediaPipe’s upper-body tracking with the Kinect’s RGB and depth streams, enabling real-time detection of the shoulder, elbow, and wrist joints. He developed the geometric algorithm used to infer hand location beyond the wrist and used this to create a dynamic bounding box that tracked the hand’s motion across frames. He also implemented the 3D coordinate projection from 2D pixel locations using the Kinect’s depth map and camera intrinsics. Additionally, he collaborated with Ross to filter and smooth depth data to ensure stability in the robotic arm’s motion.

Ross Goldbaum led the development of the robotic arm’s hardware and control system. He implemented the inverse kinematics pipeline that translated 3D hand positions into servo joint angles and developed the serial communication protocol that interfaced the main pipeline with the Arduino microcontroller. Ross fine-tuned servo control using alpha smoothing and experimented with PID logic to achieve smooth and accurate arm movement. He also designed the physical wiring and layout of the robotic system, ensuring safe and consistent actuation of the arm’s multiple degrees of freedom. His work was essential in bridging the software pipeline with tangible, responsive robotic behavior.